

Node.js and Express.js

- **transpiled** ==> *Translate + Compile* = Transpile (Source-to-Source Compiler).
- Transpiling means taking source code written in one version/syntax of a language and translating it into an older or standard version of the SAME language so that older software can understand it.
- Real-World Examples in Web Development:
 - Transpiling JSX ==> Plain JavaScript
 - Web browsers do NOT understand JSX (like

Hello {name}

).

- A transpiler (**like *Babel* or *Vite***) takes your JSX and translates (transpiles) it into standard JavaScript:

```
// What you write:
<h1>Hello {name}</h1>

// What the transpiler turns it into for the browser:
React.createElement('h1', null, `Hello ${name}`)
```

- Transpiling Modern JS (ES2024) ==> Older JS (ES5)
- Transpiling vs Compiling (The Difference):

Compiling: Taking human code (like C++ or Rust) and *turning it into Machine Code / 0s and 1s (.exe)*.

Transpiling: Taking human code (like JSX or TypeScript) and *turning it into another human code (Standard JavaScript)!*

- `npm init -y` (skips all que's) to initialize the **package.json**, here I am building a backend from **bare-metal scratch** with **no templates!**
- then create `index.js` and **Configure package.json** with **"start": "node index.js"**.

Simple web server

- First Server :-

```
const http = require('http')
```

```
const app = http.createServer((request, response) => {
  response.writeHead(200, { 'Content-Type': 'text/plain' })
  response.end('Hello World')
})

const PORT = 3001
app.listen(PORT)
console.log(`Server running on port ${PORT}`)
```

1. const http = require('http')

1. http is a built-in module inside Node.js.

2. **require()** is the older **Node.js** way of importing packages (**it's the exact same concept as import http from 'http'**).

2. The (request, response) Callback

```
http.createServer((request, response) => { ... })
```

- Every web server in the world operates on *Request* and *Response*:
 - **request**: Contains everything the user sent (the **URL** they clicked, **the data**, **headers**).
 - **response**: The *pipe you use to send data BACK to the user's browser*.
 - response.writeHead(200, { 'Content-Type': 'text/plain' }): Tells the browser: "Status is 200 OK, and I am sending you plain text."
 - response.end('Hello World'): Sends "Hello World" and closes the connection!
-

3. app.listen(PORT) (The Doorbell)

```
const PORT = 3001
app.listen(PORT)
```

- this is just basic understanding of how we make server live. but very traditional method
-

Express-Js

- Express is a lightweight library that removes 90% of the painful boilerplate of Node's raw http module.
- **Transitive Dependencies** ((The dependencies of your dependencies!)) :

- The creators of Express didn't write every single piece of code from scratch. Express relies on smaller helper packages (like cookie, accepts, content-type, qs). And those helper packages rely on other smaller helper packages!
 - Your Project ==> Express ==> helper-package ==> sub-helper-package
 - semantic versioning
 - Caret symbol ^
 - Major . Minor . Patch
 - understand by e.g.
 - Major - 4.18.2 ==> 5.0.0
 - Minor - 4.18.2 ==> 4.19.0
 - Patch - 4.18.2 ==> 4.18.3
-

Web and Express

- First Server with **Express**:

```
const express = require('express'); // import express from 'express';
const app = express();

let notes = [
  {
    id: "1",
    content: "HTML is easy",
    important: true
  },
  {
    id: "2",
    content: "Browser can execute only JavaScript",
    important: false
  },
  {
    id: "3",
    content: "GET and POST are the most important methods of HTTP protocol",
    important: true
  }
];

app.get('/', (request, response) => {
  response.send('<h1>Hello Vinayak, Welcome to NodeJS World!</h1>');
});

app.get('/api/notes', (request, response) => {
  response.json(notes)
});
```

```
const PORT = 3001;

app.listen(PORT, () => {
  console.log(`Server Running on Port : http://localhost:${PORT}`)
});
```

- we define two routes to the application.
 - The first one defines an event handler that is used to handle *HTTP GET requests made to the application's / root*:

```
app.get('/', (request, response) => {
  response.send('<h1>Hello World!</h1>')
})
```

- The event handler function accepts **two parameters**.
 - The first request parameter contains all of the information of the HTTP request and the second response parameter is used to define how the request is responded to.
 - In our code, the request is answered by using the **send** method of the **response object**.
 - Express automatically sets the value of the Content-Type header to be text/html.
 - content-type :- **text/html**; charset=utf-8
 - The status code of the response defaults to 200.

-
- The second route defines an event handler that handles HTTP GET requests made to the notes path of the application:

```
app.get('/api/notes', (request, response) => {
  response.json(notes)
})
```

- The request is responded to with the *json* method of the **response object**.
 - Calling the method will send the notes array that was passed to it as a JSON formatted string.
 - Express automatically sets the **Content-Type header** with the appropriate value of **application/json**.
 - content-type :- **application/json**; charset=utf-8

-
- In the earlier version where we were only using Node, we had to transform the data into the JSON formatted string with the *JSON.stringify* method:

```
response.end(JSON.stringify(notes))
```

- With Express, this is no longer required, because this transformation happens automatically.
 - It's worth noting that JSON is a data format. However, it's often represented as a string and is not the same as a JavaScript object, like the value assigned to notes.
- `JavaScript Object` vs `JSON.stringify(object)`

```
'JavaScript Object' typeOf = object
const person = { name: "vinayak", age: 23 }

const json = JSON.stringify(person);
'json is become now json string object and it will return everything in
String format'

return {"name":"vinayak","age":23}
'JSON String Object' typeOf = string
```

Automatic Change Tracking

- kill old session of the server and restart the server on the change of single line of code into index.js
 - configure script in **package.json**
 - **"dev":"node --watch index.js"**
 - and shoot *npm run dev*

REST - Representational State Transfer, aka REST

- Let's expand our application so that it provides the same RESTful HTTP API as json-server
- If we define the resource type of note to be *notes*, then the address of a note resource with the identifier 10, has the unique address ***www.example.com/api/notes/10***.
- The URL for the entire collection of all note resources is ***www.example.com/api/notes***.
- We can execute different operations on resources. The operation to be executed is defined by the HTTP verb:

URL	verb	functionality
notes/10	Get	fetches a single resource
notes	Get	fetches all resources in the collection

URL	verb	functionality
notes	Post	creates a new resource based on the request data
notes/10	Delete	removes the identified resource
notes/10	Put	replaces the entire identified resource with the request data
notes/10	Patch	replaces a part of the identified resource with the request data

Fetching a single resource

```
app.get('/api/notes/:id', (request, response) => {
  const id = request.params.id
  const note = notes.find(note => note.id === id)
  response.json(note)
})
```

- Now `app.get('/api/notes/:id', ...)` will handle all HTTP GET requests that are of the form `/api/notes/SOMETHING`, where `SOMETHING` is an arbitrary string.
- If we search for a note with an id that does not exist, the server responds with:
 - `'/api/notes/100/'`
 - The HTTP status code that is returned is 200, which means that the response succeeded. There is no data sent back with the response, since the value of the content-length header is 0, and the same can be verified from the browser.
 - The reason for this behavior is that the `note` variable is set to `undefined` if no matching note is found.
 - The situation needs to be handled on the server in a better way. If no note is found, the server should respond with the status code 404 not found instead of 200.

```
app.get('/api/notes/:id', (request, response) => {
  const id = request.params.id
  const note = notes.find(note => note.id === id)
  if(!note){
    return response.status(404).end()
  }
  response.json(note)
})
```

- **The Golden Rule of Express Guard Clauses:**
 - In Express, `response.send()` / `response.end()` does not stop JavaScript. Always write **return response.status(...)** to exit early!

- Since no data is attached to the response, **we use the status method** for **setting the status** and the **end method for responding to the request without sending any data**.

Deleting resources

- Next, let's implement a route for deleting resources. Deletion happens by making an HTTP DELETE request to the URL of the resource:

```
app.delete('/api/notes/:id', (request, response) => {  
  const id = request.params.id  
  notes = notes.filter(note => note.id !== id)  
  
  response.status(204).end()  
})
```

- If deleting the resource is successful, meaning that the note exists and is removed, we respond to the request with the **status code 204 no content** and **return no data with the response**.
- There's no consensus on what status code should be returned to a DELETE request if the resource does not exist. **The only two options are 204 and 404**. For the sake of simplicity, our application will respond with 204 in both cases.

Receiving data

- Next, let's make it possible to add new notes to the server.
- Adding a note happens by making an **HTTP POST request** to the address `http://localhost:3001/api/notes`.
- and by sending all the information for the new note in the **request body in JSON format**.
- Process to activate the json-parser and implement an initial handler for dealing with the HTTP POST requests:

```
const express = require('express')  
const app = express()  
  
<!-- Middleware -->  
app.use(express.json())  
  
//...  
  
app.post('/api/notes', (request, response) => {  
  const note = request.body  
  console.log(note)
```

```
response.json(note)
})
```

- Why it is so important:
 - When a client sends JSON in a POST request, it arrives at the server as a **raw stream of text**.
 - `app.use(express.json())` is Express's built-in JSON Parser Middleware.
 - **It intercepts the incoming raw text**,^{**} *parses it into a real JavaScript object*^{**}, and *attaches it to `request.body`*!
 - ⚠ If you forget `app.use(express.json())`, **`request.body` will always be undefined!**

Middleware

- **`express.json()` IS a middleware!** Whenever a **request arrives**, **`express.json()` intercepts it**, **parses the raw text into a JS object**, and **passes it forward to your `app.post()` route!**

About HTTP request types

- The HTTP standard talks about two properties related to request types, **safety** and **idempotency**.
- **Safety** means that the executing request must not ***cause any side effects*** on the **server**.
- By side effects, we mean that **the state of the database must not change as a result of the request**, and the response must only return data that already exists on the server.
- **`Request(browser) => backend(processing request.body from Safety measures)`**
 - **`==> if(Found threats or irrelevant stuff)`** **`==>`** handle it on server, response back to **browser** with appropriate **Error**
 - **`==> else(Heads towards database)`**
- **"Safe Methods" :**
 - A "Safe" method is a request that **ONLY READS** data and **NEVER** modifies or destroys anything on the server.
 - GET is a Safe method because reading data never changes the database.
- **"Idempotent Methods"**
 - **"Idempotent"** (pronounced eye-dem-po-tent) means: Running a request 1 time produces the **EXACT SAME** result as running it 100 times in a row!
 - Let's look at the Elevator Button vs ATM Button analogy:
 - The Elevator Button (*Idempotent*):

- If you press the elevator button 1 time, the elevator comes to your floor. If you aggressively press the elevator button 50 times in a row, **the elevator still just comes to your floor! The end result is identical.**
 - The ATM Cash Button (*Non-Idempotent* / *POST*):
 - If you press "Withdraw \$100" 1 time, you get \$100. If you press "Withdraw \$100" 5 times, you just withdrew \$500 and your bank account is empty!
 - **GET is Idempotent:** Fetching data 10 times gives the same result.
 - **PUT is Idempotent:** If you update Arto's number to "123-456", running that exact update 10 times still leaves his number as "123-456".
 - **DELETE is Idempotent:** Deleting Note #3 leaves Note #3 deleted. Deleting it 5 more times still means Note #3 is deleted.
 - **POST is NOT Idempotent:** If you send a POST request to create "Arto" 5 times, the server will create 5 separate duplicate records with 5 new IDs!
-

Middleware FSO

- The Express json-parser used earlier is a middleware.
- Middleware are functions that can be used for handling request and response objects.
- In practice, you can use several middlewares at the same time. When you have more than one, they're executed one by one in the order that they were listed in the application code.
- Middleware is a function that receives three parameters:

```
const requestLogger = (request, response, next) => {  
  console.log('Method:', request.method)  
  console.log('Path: ', request.path)  
  console.log('Body: ', request.body)  
  console.log('---')  
  next() <===  
}
```

- At the end of the function body, the next function that was passed as a parameter is called **next**. The next function **yields control to the next middleware**.
- Middleware is used like this:

```
app.use(express.json())  
app.use(requestLogger)
```

- Middleware functions have to be used before routes when we want them to be executed by the route event handlers.
- Sometimes, we want to use middleware functions after routes. We do this when the middleware functions are only called if no route handler processes the HTTP request.
 - This middleware will be used for catching requests made to non-existent routes

```
const unknownEndpoint = (request, response) => {
  response.status(404).send({ error: 'unknown endpoint' })
}

app.use(unknownEndpoint)
```

Middleware by Mentor

- Imagine an Express server as a Factory **Conveyor Belt** => automatic vertical elevator/pusher
- Whenever an **HTTP request arrives from a browser or Postman, it is placed** on the **conveyor belt** and **passes through** several Stations (**Middleware Functions**) in order from top to bottom:

```
[ Request Arrives ]
  ↓

[ Station 1: express.json() ] --> (Parses raw text into request.body)
  ↓ calls next()

[ Station 2: Morgan Logger ] --> (Logs request details to terminal)
  ↓ calls next()

[ Station 3: Your Routes ] --> (app.get, app.post, app.delete)
  ↓ (If no route matched)

[ Station 4: Unknown Endpoint ] --> (Returns 404 error)
```

- Why Middleware Functions have 3 Parameters: (request, response, next)
 - Every middleware function in Express looks like this:

```
const myMiddleware = (request, response, next) => {
  console.log('A request passed through Station 1!');

  // 🚨 CRITICAL: You MUST call next()!
  next();
};

app.use(myMiddleware);
```

- What is next()?
 - next() is the "Push Conveyor Belt Forward" button!
 - Why the Order of Middleware Matters:
 - **express.json() and loggers must be at the TOP** : They need to parse and log the request before it reaches your routes.
 - **unknownEndpoint (404 catch-all) must be at the VERY BOTTOM** : If you put the 404 handler at the top, it will intercept and block every single request before your routes even have a chance to run!
-

Morgan - Logger

basic setup of morgan logger

- **npm install morgan**
- `const morgan = require('morgan');`
 - `// Use the pre-built 'tiny' format app.use(morgan('tiny'));`
- What it does:
 - Whenever you visit `http://localhost:3001/api/persons`, Morgan instantly prints this in your terminal:
 - `GET /api/persons 200 245 - 2.154 ms`
 - (Method(GET), URL(/api/persons), Status Code(200), Response Size(245), Time taken in milliseconds) - (2.154ms).
 -

Custom Morgan Token (Logging the POST Body!)

- By default, Morgan does not log the request body. But Morgan lets you create a Custom Token using **morgan.token()**!
 - How to create the custom token:

1. Create a token called 'body' that stringifies request.body:

```
// Creates a custom Morgan token named :body
morgan.token('body', (request, response) => {
  return JSON.stringify(request.body);
});
```

2. Configure Morgan using a custom format string that includes :body:

```
app.use(morgan(':method :url :status :res[content-length] -  
:response-time ms :body') );
```

- The Result in your Terminal:
 - When you send a **GET /api/persons**, it prints: **GET /api/persons 200 245 - 2.154 ms {}**
 - When you send a POST /api/persons with a new contact, it prints: **POST /api/persons 200 64 - 4.231 ms {"name":"Liisa Marttinen","number":"040-243563"}!**

Deploying app to internet

- React Frontend: `http://localhost:5173` (Origin A - Port 5173)
- Express Backend: `http://localhost:3001` (Origin B - Port 3001)
- Because the port numbers are different (5173 vs 3001), the browser's built-in Same-Origin Policy (SOP) security rule says:
 - "Warning! JavaScript running on Port 5173 is trying to steal data from Port 3001! I will BLOCK this request unless Port 3001 gives permission!"

Same origin policy - (SOP) and CORS - (Cross origin Resource Sharing)

- The issue lies with a thing called **same origin policy**. A URL's origin is defined by the combination of protocol (AKA scheme), **hostname**, and **port**.

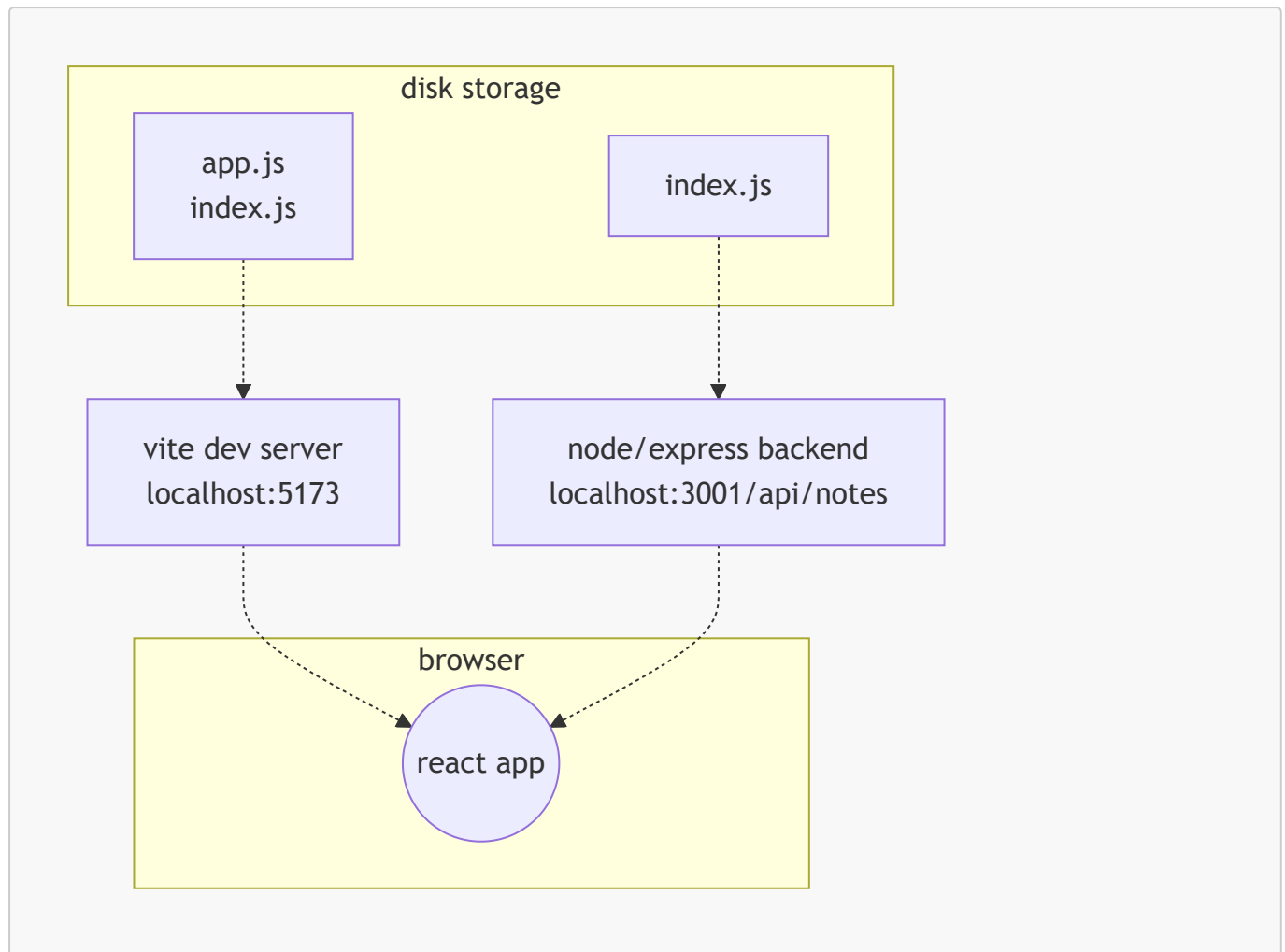
```
http://example.com:80/index.html
```

```
protocol: http  
host: example.com  
port: 80
```

- The **same-origin policy** is a *security mechanism implemented by browsers* in order to prevent **session hijacking** among other security vulnerabilities.
- In order to enable legitimate cross-origin requests (requests to URLs that don't share the same origin) W3C came up with a mechanism called CORS(Cross-Origin Resource Sharing)
 - A web page may freely embed cross-origin images, stylesheets, scripts, iframes, and videos.
- The problem is that, by default, the JavaScript code of an application that runs in a browser can only communicate with a server in the same origin. Because our server is in localhost port 3001, while our frontend is in localhost port 5173, they do not have the same origin.
- same-origin policy and CORS are not specific to React or Node. They are universal principles regarding the safe operation of web applications.
- We can allow requests from other origins by using **Node's cors middleware**.
- Same Origin, it must be the exact same machine, exact same domain/IP, and exact same port:
- take the middleware to use and allow for requests from all origins:

```
const cors = require('cors');  
app.use(cors())
```

Basic App flow



Application to the Internet

- Render (Cloud Hosting Platforms)

// 🚧 Right now you have: `const PORT = 3001;`

// ✅ Change it to this (so Render can pick its own port in the cloud!): `const PORT = process.env.PORT || 3001;`

Frontend production build (Minified code)

- What happened to your nice React code?
 - When you write React, you write clean code with comments, spaces, and nice variable names like:

```
const handleCreateNewPerson = (newPersonObject) => { ... }
```

- When you run `npm run build`, the build tool (Vite / Rollup) does 3 aggressive things.
 - **Deletes 100% of spaces, newlines, and comments:** Squeezes thousands of lines into 1 continuous line.

- **Shortens all variable names:** Renames handleCreateNewPerson to e, newPersonObject to r, and response to t.
- **Bundles 50 files into ONE single file:** Combines React, Axios, and all your components together.
- Why on earth does it do this?
 - SPEED & BANDWIDTH! 🚀
 - Your original human-readable code was 500 Kilobytes.
 - The minified alien code is only 30 Kilobytes!
- Humans NEVER read or edit minified code!
 - You write human code in src/.
 - Vite creates the alien code in dist/ automatically for the browser!
 - demo :-

```
!function(e){function r(r){for(var n,f,i=r[0],l=r[1],a=r[2],c=0,s=[
];c<i.length;c++)f=i[c],o[f]&&s.push(o[f][0]),o[f]=0;for(n in
l)Object.prototype.hasOwnProperty.call(l,n)&&
(e[n]=l[n]);for(p&&p(r);s.length;)s.shift();return u.push.apply(u,a||
[]),t()}function t(){for(var e,r=0;r<u.length;r++){for(var
t=u[r],n=!0,i=1;i<t.length;i++){var l=t[i];0!==o[l]&&(n=!1)}n&&(u.splice(r-
,1),e=f(f.s=t[0]))}return e}var n={},o={2:0},u=[];function f(r)
{if(n[r])return n[r].exports;var t=n[r]={i:r,l:!1,exports:{}};return
e[r].call(t.exports,t,t.exports,f),t.l=!0,t.exports}f.m=e,f.c=n,f.d=function(
e,r,t){f.o(e,r)||Object.defineProperty(e,r,
{enumerable:!0,get:t})},f.r=function(e){"undefined"!==typeof
Symbol&&Symbol.toStringTag&&Object.defineProperty(e,Symbol.toStringTag,
{value:"Module"})}
```

Rest All explain by mentor FSO hits with heavy Theroy

1. Serving Static Files: app.use(express.static('dist')) 📁

- What is a "Static File"?

Files that don't change on the server: your compiled React index.html, main.js, and style.css sitting inside the dist folder.

- What does app.use(express.static('dist')) do?

When someone opens http://localhost:3001 in Chrome:

- Express looks inside the dist folder.
- It finds index.html and hands it to the browser.

- Result: Your entire React application opens directly from your Express server on Port 3001!

2. Relative URLs: baseUrl = '/api/persons'

- In Part 2 (Development):
 - You had to write the full address:

```
const baseUrl = 'http://localhost:3001/api/persons';
```

- In Production:
 - Because React and Express are now running on the exact same server, you don't need `http://localhost:3001` anymore! You change it to a Relative URL:

```
const baseUrl = '/api/persons'; // 👉 Just the path!
```

- Why Relative URLs are magic:
 - On your laptop, the browser automatically fetches: `http://localhost:3001/api/persons`.
 - When deployed on Render in the cloud, the browser automatically fetches: `https://your-app.onrender.com/api/persons`!
 - You never have to change the URL again!

3. The Development Proxy in Vite (vite.config.js)

- Here is the developer dilemma: If we change `baseUrl` in React to `'/api/persons'`, what happens when you run `npm run dev` in React on port 5173?
 - The browser tries to fetch `http://localhost:5173/api/persons`.
 - 404 Error! Because Vite on 5173 is just a frontend tool—it doesn't have your persons database!
- The Solution (The Proxy):
 - In your React project's `vite.config.js`, you add a Proxy rule:

```
server: {  
  proxy: {  
    '/api': {  
      target: 'http://localhost:3001',  
      changeOrigin: true,  
    },  
  },  
}
```



```
} ,  
}
```

- What this does: When you are coding on port 5173 and React asks for `/api/persons`, Vite secretly forwards that request to `http://localhost:3001` behind the scenes!

Result : You can keep `baseUrl = '/api/persons'` in React forever, and it works in BOTH Development and Production!

4. Deploying to Render.com 🌩

- When you deploy your phonebook-backend to Render:
 - Render runs `npm install` to download Express and CORS.
 - Render runs `npm start` (node `index.js`).
 - Express serves the `dist/` folder (your React UI) and responds to `/api/persons` (your API).
 - Render gives you a live global link (e.g. `https://phonebook-backend.onrender.com`) that works on any phone in the world!
-

Summary of the Whole Picture:

- React (part2/phonebook): Set `baseUrl = '/api/persons'` →→ Run `npm run build` →→ creates `dist/`.
- Backend (phonebook-backend): Copy `dist/` into backend →→ Add `app.use(express.static('dist'))` →→ Push to GitHub.
- Render: Connect GitHub repo →→ Click Deploy →→ Your full-stack app is live on the internet!

Saving data to MongoDB

Debugging Node applications

- Ways to Debug Node.js Apps:

- `console.log()` (The #1 Tool used by 95% of Engineers):

```
console.log('Incoming Body:', request.body);  
console.log('Found Person:', person);
```

- Chrome DevTools for Node (**`node --inspect index.js`**):

- You can run Node with `--inspect` and open `chrome://inspect` in your browser to debug Node backend code using Chrome's DevTools console!

- When bugs occur, the worst of all possible strategies is to continue writing code. It will guarantee that your code will soon have even more bugs, and debugging them will be even more difficult
-

MongoDB

- The reason for using Mongo as the database is its lower complexity compared to a relational database.
 - Document databases differ from relational databases in how they organize data as well as in the query languages they support. Document databases are usually categorized under the NoSQL umbrella term.

Collection and Documents

- The Folder & File Analogy 📁
- Think of MongoDB like a file cabinet on your computer:

```
MongoDB Database (The Whole Cabinet)  
├──  
└── Collection (A Folder)  
    ├──  
    └── Document (A Single JSON File inside the folder)
```

1. What is a Document? (A Single Item)

- A Document is just ONE individual record stored in JSON (BSON) format.

```
{  
  "_id": "64f1a2b3c4d5e6f7a8b9c0d1",  
  ...  
}
```

```
"name": "Arto Hellas",  
"number": "040-123456"  
}
```

2. What is a Collection? (The Group / Array)

- A Collection is simply a group or list of documents (equivalent to an array of objects in JavaScript).
- Example:

```
const persons = [  
  {name : "Arto Hellas", number : 040-123456},  
  {name : "Arto Hellas", number : 040-123456},  
  {name : "Arto Hellas", number : 040-123456},  
  {name : "Arto Hellas", number : 040-123456}  
]
```

- The whole **array persons** is called the **Collection**.
- ****Each individual object ****{ id: "1", ... } inside that array is **called a Document**.

Process of db connection

- create project first ==> cluster(db) ==> set_user (name,password,role) => IP address list allow for all by (0.0.0.0/0) else only specific for you by default already added you IP add.
- **install mongoose** into backend folder
- mkdir file called mongo.js init all requirements into this file.
- Connection code
- code

```
const mongoose = require('mongoose');  
  
// 1. Check if the user passed a password in the terminal command  
if (process.argv.length < 3) {  
  console.log('give password as argument');  
  process.exit(1);  
}  
  
// 2. Read the password from the 3rd command-line argument (process.argv[2])  
// node mongo myPassword123 [3]  
const password = process.argv[2];  
  
// 3. The MongoDB Connection String (Points to your Atlas cloud database)  
const url = `mongodb+srv://fso-vinayak:${password}@notes-  
app.loz4osq.mongodb.net/notes-app?appName=notes-app`
```

```
mongoose.set('strictQuery', false);

// 4. Open the connection to the cloud database
console.log('Connecting to MongoDB Atlas...');
mongoose.connect(url);

// Define the Schema (The Blueprint of what a Note looks like)
const noteSchema = new mongoose.Schema({
  content: String,
  important: Boolean
});

// Model Constructor Capitalization: Since mongoose.model(...) returns a
// Constructor Function (a factory to create new objects), convention dictates
// using PascalCase:

// 6. Create the Model (The Constructor that talks to the 'notes' collection)

const Note = mongoose.model('Note', noteSchema);

// If you ever desperately wanted it to be named persons instead of
// people, Mongoose lets you pass a custom collection name as the 3rd argument:
// mongoose.model('Person', personSchema, 'persons'),
// but letting Mongoose do its default people is standard practice

// 7. Create a new Note document object
const note = new Note({
  content: 'HTML is easy and fun',
  important: true
});

// 8. Save the note to the cloud database and CLOSE the connection
note.save().then(result => {
  console.log('note saved');
  mongoose.connection.close(); // 👉 Must close the connection so the
  // script exits!
});
```

◦ Explanation :-

```
const mongoose = require('mongoose');
```

1. The Command-Line Argument: **process.argv**

```
if(process.argv.length < 3){
  console.log('give password as argument');
```

```
    process.exit(1);
  }
  <!-- Read the password from the 3rd command-line argument (process.argv[2]) -
  ->
  const password = process.argv[2];
```

- **When you type in terminal: node mongo.js myPassword123**
- Node creates an array called process.argv:
 - [node ,mongo.js ,myPassword123]
 - 1. process.argv[0] = Path to Node engine (node)
 - 2. process.argv[1] = Path to the file (mongo.js)
 - 3. process.argv[2] = Whatever you typed 3rd: 'myPassword123'!
- If you forgot to type the password, process.argv.length is less than 3, so process.exit(1) immediately stops the script with an error code!

2. The Connection URL & mongoose.connect(url)

```
<!-- The MongoDB Connection String (Points to your Atlas cloud database) -->

const url = `mongodb+srv://fso-vinayak:${password}@notes-
app.loz4osq.mongodb.net/notes-app?appName=notes-app`

mongoose.set('strictQuery', false);
mongoose.connect(url)
mongoose.connect(url,{family:4})
```

- **url**: Embeds your dynamic password and tells Mongoose to connect to the noteApp database.
- *strictQuery*: **true**: Mongoose strictly ignores any search field that is not written in your Schema.
- *strictQuery*: **false**: Mongoose allows flexible searching in the database and silences annoying terminal warning messages.
- **mongoose.connect(url)**: Opens the network TCP socket connection across the internet to the MongoDB Atlas servers.
- Passing { family: 4 } tells Mongoose:
 - "Hey Mongoose, force the connection to strictly use IPv4 so my connection never hangs or lags!"
 - If your mongo.js connected in 1 second without errors (which it did on your screenshot!), your network is already working fine!.

- But if you ever run into a weird connection timeout error on a different Wi-Fi, adding { family: 4 } is a handy networking fix.

The Schema (The Architectural Blueprint)

```
const noteSchema = new mongoose.Schema({
  content: String,
  important: Boolean,
});
```

- MongoDB by itself is **"schemaless"** (*it blindly accepts any garbage JSON*).
- Mongoose Schemas give your app strict rules:
 - Every note MUST have content as a String and important as a Boolean.

4. The Model (The Constructor) & Mongoose Naming Magic

```
const Note = mongoose.model('Note', noteSchema);
```

- Note is now a JavaScript Constructor function (like a Class) that lets us create, find, update, and delete notes.
- **Mongoose's Secret Pluralization Magic** : When you name your model singular capitalized 'Note', Mongoose automatically lowercases and pluralizes it to name the collection in the database notes! (e.g., 'Person' becomes 'people' or 'persons', 'User' becomes 'users').

Creating & Saving the Document (Async Promise!)

```
const note = new Note({
  content: 'HTML is easy and fun',
  important: true,
});

note.save().then(result => {
  console.log('note saved!');
});

mongoose.connection.close(); // 🙋 WHY IS THIS HERE?
});
```

- new Note({ ... }) creates the document object in RAM.
- note.save() sends the insert command over the internet to MongoDB Atlas on disk.

- **note.save() returns a PROMISE!** When MongoDB confirms the data is saved, the `.then()` callback runs!

-
- ⚠ Why `mongoose.connection.close()` is Mandatory in CLI scripts:
 - Node.js keeps the database network connection open forever by default.
 - If you do NOT call `mongoose.connection.close()`, your script will finish saving, but your terminal will hang and freeze forever, waiting for more commands!
 - Calling `mongoose.connection.close()` cuts the network wire so your terminal returns cleanly to the command line prompt!
-

Now: How to FETCH / READ data from MongoDB (`Note.find()`);

- Now let's see how Mongoose reads all notes from the database.
- In `mongo.js`, you can replace the save code with **`Note.find({})`**:
 - What `{}` means in **`Note.find({})`**:
 - **The empty object `{}` is a filter search query that means "Find EVERYTHING without restrictions!"**
 - **If you wanted only important notes, you would write: `Note.find({ important: true })`.**

```
Note.find({}).then(result =>{
  result.forEach(note =>{
    console.log(note);
  });
  mongoose.connection.close().
});
`
```

Connection the backend to a Database

```
const mongoose = require('mongoose');

const url = process.env.MONGODB_URI
mongoose.set('strictQuery', false);

console.log('Connecting to MongoDB Atlas...');

mongoose.connect(url)
  .then(() => {
    console.log('connected to MongoDB');
  })
```

```
.catch(error => {
  console.log('error connecting to MongoDB:', error.message);
})

const noteSchema = new mongoose.Schema({
  content: String,
  important: Boolean
});

noteSchema.set('toJSON', {
  transform: (document, returnedObject) => {
    returnedObject.id = returnedObject._id.toString()
    delete returnedObject._id
    delete returnedObject.__v
  }
});

const Note = mongoose.model('Note', noteSchema);

module.exports = Note;
```

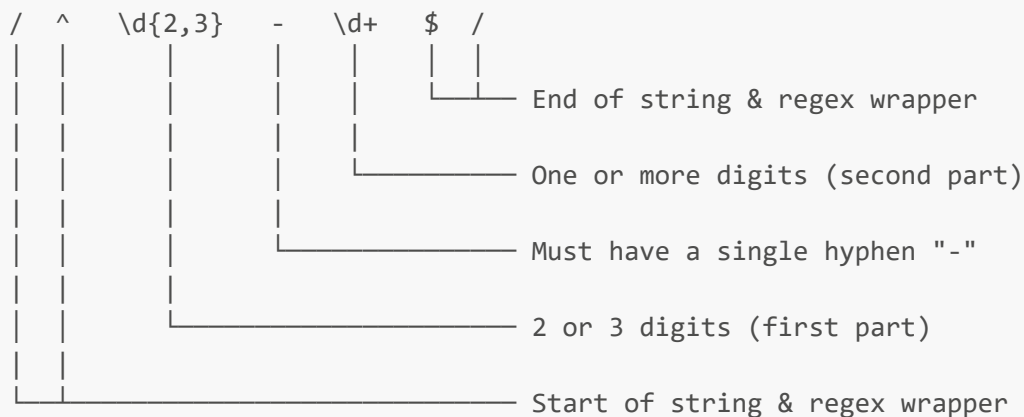
URI Vs URL

- URI = Uniform Resource Identifier (The general identity)
- URL = Uniform Resource Locator (The specific location / address)
- **The Rule: Every URL is a URI, but not every URI is a URL.**

Validation and ESLint

Validation in regex for phonenumber

- eg. 09-1234556 and 040-22334455 are valid phone numbers
- eg. 1234556, 1-22334455 and 10-22-334455 are invalid



Regex Breakdown & Reference Guide

1. Breakdown of `/^\\d{2,3}-\\d+$/`

Symbol	Name	Meaning	Example Match
<code>/ ... /</code>	Delimiters	Encloses the regular expression in JavaScript	<code>/pattern/</code>
<code>^</code>	Start Anchor	Asserts the start of the string (no leading characters allowed)	Starts right at beginning
<code>\\d</code>	Digit Class	Matches any single digit from <code>0</code> to <code>9</code>	<code>0, 4, 9</code>
<code>{2,3}</code>	Range Quantifier	Matches the preceding token between 2 and 3 times	<code>09</code> (2 digits), <code>040</code> (3 digits)
<code>-</code>	Literal Character	Matches the exact hyphen character <code>-</code>	<code>-</code>
<code>\\d+</code>	Plus Quantifier	Matches 1 or more digits	<code>1234556, 22334455</code>
<code>\$</code>	End Anchor	Asserts the end of the string (no trailing characters allowed)	Ends right after digits

3. General Regex Cheat Sheet

Token	Meaning	Example
<code>\d</code>	Any digit <code>[0-9]</code>	<code>\d</code> matches <code>7</code>
<code>\D</code>	Any non-digit	<code>\D</code> matches <code>a</code> , <code>@</code>
<code>\w</code>	Any word character <code>[a-zA-Z0-9_]</code>	<code>\w</code> matches <code>A</code> , <code>3</code> , <code>_</code>
<code>\s</code>	Any whitespace (space, tab, newline)	<code>\s</code> matches
<code>.</code>	Any character (except newline)	<code>.</code> matches <code>x</code> , <code>9</code> , <code>%</code>
<code>*</code>	0 or more times	<code>a*</code> matches <code>""</code> , <code>"a"</code> , <code>"aaa"</code>
<code>+</code>	1 or more times	<code>a+</code> matches <code>"a"</code> , <code>"aaa"</code>
<code>?</code>	0 or 1 time (optional)	<code>https?</code> matches <code>http</code> , <code>https</code>
<code>{n}</code>	Exactly <code>n</code> times	<code>\d{4}</code> matches <code>2026</code>
<code>{n,m}</code>	Between <code>n</code> and <code>m</code> times	<code>\d{2,4}</code> matches <code>99</code> , <code>9999</code>
<code>[abc]</code>	Any character inside brackets	<code>[abc]</code> matches <code>a</code> , <code>b</code> , or <code>c</code>
<code>[^abc]</code>	Any character NOT inside brackets	<code>[^abc]</code> matches <code>x</code> , <code>1</code>
<code>(a b)</code>	Either <code>a</code> or <code>b</code>	<code>(cat dog)</code> matches <code>cat</code> or <code>dog</code>

4. Usage in Mongoose Schema

```
const personSchema = new mongoose.Schema({
  name: {
    type: String,
    minLength: 3,
    required: true
  },
  number: {
    type: String,
    minLength: 8,
    required: true,
    validate: {
      validator: function(v) {
        return /^\\d{2,3}-\\d+$/\\.test(v);
      },
      message: props => `\\${props.value} is not a valid phone number!`
    }
  }
});
```

⚙️ How `.test(v)` works:

validator: function(v) { return /^\d{2,3}-\d+\$/ .test(v) }

- v is the value the user typed in the phone number box.
 - .test(v) runs the regex.
 - If it matches the pattern → returns true (Mongoose saves it).
 - If it does NOT match → returns false (Mongoose rejects it with your custom message!).
-

1. What is ESLint? (The Big Picture)

- ESLint is the automated Style & Quality Inspector for JavaScript:
 - ESLint is the automated Style & Quality Inspector for JavaScript:
 - It reads your code without running it.
 - It catches hidden bugs (like unused variables or accidental global variables).
 - It enforces clean formatting rules (e.g. "In this company, we always use single quotes and 2 spaces")
-

The 4 Simple Steps to Set Up ESLint from Scratch:

- Step 1: install the Tools
 - **`npm install eslint @eslint/js globals @stylistic/eslint-plugin-js --save-dev`**

```
* eslint: The main inspection engine.

* @eslint/js: Recommended default JavaScript rules.

* globals: Tells ESLint that Node variables (like process.env and
require) are allowed.

* @stylistic/eslint-plugin-js: Rules for visual formatting (spaces,
quotes, semicolons).

* --save-dev: Tells npm this is only a development tool (not needed in
live production).
```

- Step 2: The Rules File (eslint.config.mjs)
 - You create a single file named **eslint.config.mjs** in your project root. Think of this file like a Rulebook:

```

import globals from "globals";
import js from "@eslint/js";
import stylisticJs from "@stylistic/eslint-plugin-js";

export default [
  js.configs.recommended, // 1. Use standard recommended JS rules
  {
    files: ["**/*.js"],
    languageOptions: {
      sourceType: "commonjs", // 2. We use CommonJS (require/exports)
      globals: {
        ...globals.node, // 3. Allow Node globals (process, require,
etc.)
      },
      ecmaVersion: "latest",
    },
    plugins: {
      "@stylistic/js": stylisticJs,
    },
    rules: {
      "@stylistic/js/indent": ["error", 2], // Rule: Indent with 2
spaces
      "@stylistic/js/quotes": ["error", "single"], // Rule: Use single quotes
, ,
      "@stylistic/js/semi": ["error", "never"], // Rule: No semicolons at
end of lines
      "eqeqeq": "error", // Rule: Always use ===
instead of ==
      "no-trailing-spaces": "error", // Rule: No useless spaces
at end of lines
      "object-curly-spacing": ["error", "always"], // Rule: { name: 'Dan' }
instead of {name:'Dan'}
      "arrow-spacing": ["error", { "before": true, "after": true }],
      "no-console": "off", // Rule: console.log is
allowed in backend!
    },
  },
  {
    ignores: ["dist/**", "node_modules/**"], // Rule: Do not inspect
the frontend build or node_modules!
  },
];

```

- 📄 Step 3: Add the lint command in package.json

```

"scripts": {
  "start": "node index.js",

```

```
"dev": "node --watch index.js",  
"lint": "eslint ." // <=====  
}
```

-  Step 4: Run the Inspector!
 - npm run lint

Magic of eslint

1. ◦ ESLint tells you what to fix:
- When you run npm run lint, ESLint acts like a helpful guide.

```
index.js:15:3  error  Expected indentation of 2 spaces but found 4  
@stylistic/js/indent  
  
index.js:20:10 error  Strings must use singlequote  
@stylistic/js/quotes
```

- It tells you the exact file, exact line number, and what rule needs fixing.

-
2. ◦ Auto-Fixing (The Magic Trick! ✨):

run:-

```
npx eslint . --fix
```

- ESLint will automatically fix spaces, quotes, and semicolons across your entire project in 1 second!